Jugend Forscht Junior

tPlug – Der intelligente Multifunktionsschalter

Tim Jockers



email: tim.jockers@icloud.com

März 2026 (überarbeitete Version)

Inhaltsverzeichnis

1 Fachliche Kurzfassung **1**

2 Motivation und Fragestellung **1**

2.1 Anwendungsbeispiele im Alltag 2

2.1.1 Beleuchtung 2

2.1.2 Lüftung 2

2.1.3 Warmwasserpumpe 2

3 Hintergrund und theoretische Grundlagen **3**

3.1 Funktionsweise und Design der Bedienungsoberfläche 3

3.2 Theoretische Grundlagen der Software 3

3.3 Funktionsweise der Sensoren 4

4 Vorgehensweise, Materialien und Methoden **4**

4.1 Die Bedienungsoberfläche 6

4.2 Elektronik des tPlugs 9

4.3 Das Gehäuse des tPlugs 12

5 Ergebnisse **13**

6 Umgesetzte Erweiterungen **14**

7 Fazit und Ausblick **14**

A Bedingungsanleitung **15**

Literatur **16**

1 Fachliche Kurzfassung

In meinem Projekt entwickle ich ein Schaltermodul, das universell im Haushalt einsetzbar ist. Mehrere Sensoren ermöglichen, dass die unterschiedlichsten Geräte optimal gesteuert werden können, sodass im Sinne der Nachhaltigkeit unnötige Laufzeiten von elektrischen Geräten verhindert werden. Beispielsweise könnte ein Lüfter in Abhängigkeit von Raumtemperatur und Luftfeuchtigkeit automatisch geschaltet werden. Mit dem selben Stecker kann aber auch eine Außenbeleuchtung über Bewegungsmelder und Zeitschaltuhr angesteuert werden. Mit einem bedienerfreundlichem TouchDisplay werden Einstellungen vorgenommen und die aktuellen Messwerte der Sensoren übersichtlich und intuitiv dargestellt. Mein interdisziplinäres Projekt vereint mein technisches Interesse für elektronische Schaltkreise und Mikrocontroller mit deren Programmierung. Die Software (einschließlich der Bedienungsoberfläche des TouchDisplays) programmiere ich in der objektorientierten Compilersprache C++.

2 Motivation und Fragestellung

Wir haben bei uns zuhause eine Warmwasserpumpe, die das Wasser vorlaufen lässt, damit es dann wenn man es braucht (z.B. zum Spülen oder zum Duschen) sofort schön warm ist. Meine Mutter erwähnte eines Tages, dass es vorteilhaft wäre, wenn man nicht zweimal in die Waschküche laufen müsste, einmal um die Pumpe anzuschalten und noch einmal um sie auszuschalten, sondern wenn sie einfach nach 10 Minuten von selber wieder ausgehen würde. Ich habe recherchiert, dass es bereits viele verschiedene Zeitschaltuhren gibt, doch ich wollte einen Schaltstecker bauen, der flexibel einsetzbar ist. So plante ich, Sensoren für Luftfeuchtigkeit, Temperatur und Bewegung zu integrieren, sowie Kalender, Zeitspannen und Timerfunktionen zu programmieren. Irgendwann hatte ich die Idee, diesen eigens erfundenen Schaltstecker tPlug zu nennen.

2.1 Anwendungsbeispiele im Alltag

Da der beschriebene Schaltstecker viele Sensoren besitzt, ergeben sich zahlreiche Anwendungen im Alltag, sodass elektrische Geräte (auch ältere) intelligent gesteuert werden können. Das reduziert die Laufzeiten von elektrischen Geräten und spart somit Energie. Sowohl die Modernisierung älterer Geräte, als auch das Einsparen von Energie, ist im Sinne der Nachhaltigkeit. Im Folgenden möchte ich einige Anwendungsbeispiele für den tPlug beschreiben.

2.1.1 Beleuchtung

Es gibt mehrere Möglichkeiten, eine Beleuchtung mit dem tPlug zu realisieren. Eine Möglichkeit ist, den Strom für die Beleuchtung – wie bei einem normalen Lichtschalter – durch einen Knopf auf dem Display anzuschalten, und ihn auch wieder durch einen Knopf auszuschalten. Zahlreiche andere Möglichkeiten ergeben sich jedoch dadurch, dass man den Strom durch verschiedene Zeiteinstellungen (Zeitpunkte sowie Zeitspannen) und zusätzlich mit einem Bewegungsmelder kombinieren kann. Nützlich ist zum Beispiel eine Außenbeleuchtung, die nur im Dunkeln auf Bewegung reagiert und nach einer Minute automatisch erlischt.

2.1.2 Lüftung

Man kann mit dem tPlug auch eine Lüftung, beispielsweise in einem Bad, schalten. Sinnvoll ist hier, dass die Lüftung bei zu hoher Temperatur oder Luftfeuchtigkeit automatisch angeht und wieder ausgeht, wenn die gewünschten Werte erreicht sind.

2.1.3 Warmwasserpumpe

Auch eine Warmwasserpumpe lässt sich perfekt mit dem tPlug steuern – das war ja auch die ursprüngliche Motivation meines Projekts. Man könnte den Strom für die Pumpe aus einer Kombination aus Kalender (Wochentage oder Intervall, z.B. jeden 3. Tag) und Uhrzeit schalten. Unter der Woche möchte ich schließlich um 7 Uhr, am Wochenende aber erst etwas später sofort warmes Wasser zum Duschen haben.

Selbstverständlich kann die Pumpe auch manuell auf Knopfdruck angehen und von selbst anhand von Uhrzeit oder Timer abschalten.

3 Hintergrund und theoretische Grundlagen

3.1 Funktionsweise und Design der Bedienungsoberfläche

Um eine benutzerfreundliche und intuitive Bedienung des tPlugs zu ermöglichen, habe ich mich entschieden, die Einstellungen des Geräts über ein TouchDisplay umzusetzen. Man unterscheidet zwischen resistiven und kapazitiven TouchDisplays [**?**]. Resistive TouchDisplays reagieren auf Druck und sind preiswerter im Vergleich zu kapazitiven TouchDisplays. Kapazitive TouchDisplays ermitteln die Eingabe durch die Veränderung der elektrischen Kapazität der Berührungsoberfläche. Derartige TouchDisplays erlauben oft komplexere Eingabemöglichkeiten durch Multitouchfunktionen und sind aber teurer in der Anschaffung. Für mein Projekt habe ich mich für die einfachere und preiswertere resistive TouchDisplay-Variante entschieden, weil diese für die von mir entworfene Bedienungsoberfläche ausreichend ist.
Die Anzeige des TouchDisplay wird über die I^2C Schnittstelle [**?**] des Mikrocontrollers angesteuert und die Touchkoordinaten über GPIOs (General Purpose Input/Output) [**?**] des Mikrocontrollers eingelesen.
Die auf dem TouchDisplay realisierte Oberfläche soll einfach bedienbar, anschaulich und intuitiv sein. Dabei habe ich mich nach den Grundsätzen der Gestaltung von Bedienungsoberflächen orientiert [**?**].

3.2 Theoretische Grundlagen der Software

Die von mir entwickelte Bedienungsoberfläche basiert auf der objektorientierten Compilersprache C++ [**?**]. Ich habe mich für eine objektorientierte Sprache entschieden, um die Umsetzung der Bedienungsoberfläche effizient zu realisieren. In einer objektorientierten Sprache werden Datenstrukturen zusammen mit Routinen in Klassen vereint. Diese Klassen können mittels des Vererbungsprinzips Schritt für Schritt von

abstrakten Datenplatzhaltern zu anwendungsspezifischen Datenstrukturen spezifiziert werden [**?**]. Dieses objektorientierte Prinzip erlaubt somit eine organisierte Programmierung von Bedienungsoberflächen. Die Programmierung der Bedienungsoberfläche basiert auf den objektorientierten Klassen des OpenSource Oberflächenpakets Qt [**?**], welche ich für meine Zwecke weiterentwickelt habe.
Da die Anzeige und Eingabe am TouchDisplay, das Auslesen der Sensoren und das Schalten des Relais unabhängig voneinander gesteuert werden muss, besteht das von mir entwickelte Programm aus mehreren Threads, das heißt Programmteilen, die parallel ablaufen. Die Verwaltung der Threads lässt sich gut mit C++ realisieren [**?**, **?**]. Für mein Projekt habe ich mich mit C++ bewusst für eine Compilersprache entschieden, damit die von mir programmierte Bedienungsoberfläche auf dem Mikrocontroller Raspberry Pi Zero mit einer relativ geringen Rechenleistung zügig und ohne merkbare Verzögerungen ausgeführt werden kann, was bei einer Umsetzung mit einer objektorientierten Interpretersprache, wie z.B. Python, nicht garantiert wäre.

3.3 Funktionsweise der Sensoren

Der Schaltstecker soll mit den eingebauten Sensoren möglichst viele Umgebungsparameter auswerten. Für die eingangs beschriebenen Anwendungen plante ich den Einbau eines Bewegungsmelders, sowie eines Temperatur- und Luftfeuchtigkeitssensors. Der Bewegungssensor reagiert auf die Infrarotstrahlung von sich bewegenden wärmeausstrahlenden Personen oder Tieren [**?**]. Die Temperatur kann über die temperaturabhängige Leitfähigkeit eines Widerstands elektronisch ermittelt werden [**?**]. Die Luftfeuchtigkeit kann durch die Veränderung der elektrischen Kapazität einer Polymerschicht elektronisch bestimmt werden [**?**]. Eine derartige Messung der Temperatur und der Luftfeuchtigkeit sind in dem Kombisensor BME280 realisiert [**?**].

4 Vorgehensweise, Materialien und Methoden

Ich habe zunächst die Software für den Mikrocontroller am Computer entwickelt. Um die Software am Computer vollständig testen zu können, simulierte ich Messwerte für

das entwickelte Programm, da am Computer keine Sensoren angeschlossen sind. Die erste Version der von mir am Computer entwickelten Software habe ich dann auf den Mikrocontroller Raspberry Pi Zero übertragen.

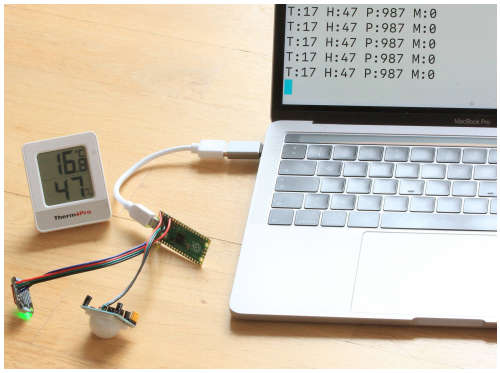


Abbildung 4.1: *Eichung des Umweltsensors und Test des Mikrocontrollers Raspberry Pi Pico.*

Die Sensoren baute ich mit dem zweiten Mikrocontroller Raspberry Pi Pico zu einem elektronischen Schaltkreis zusammen, um deren Funktionsweise testen und die ausgelesenen Sensordaten eichen zu können. Die Ansteuerung des Umweltsensors erfolgt über I^2C [**?**] und der Bewegungsmelder wird mittels GPIOs (General Purpose Input/Output) [**?**] angesteuert. Die Eichung der Messwerte des Umweltsensors führte ich durch den Vergleich mit den Temperatur- und Feuchtigkeitsmesswerten des *ThermPro Digital Hygrometer* durch. Die Messergebnisse stimmen im Rahmen der Messgenauigkeit der Sensoren und des Hygrometers überein (siehe Abbildung **??**).
Nachdem auch die Ansteuerung der Sensoren funktioniert hat, installierte ich geeignete OpenSource Treiber für das TouchDisplay, USB und für die I^2C-Schnittstelle auf dem Raspberry Pi Zero. Das auf dem Computer entwickelte Programm musste ich noch etwas auf das TouchDisplay anpassen. Nachdem die Funktionsweise aller Komponenten gegeben war, verbaute und verlötete ich alle Bauteile endgültig für den ersten tPlug Prototyp.
Jetzt funktionierte der tPlug technisch gesehen schon, aber er war noch nicht praktisch einsetzbar, da die einzelnen Teile nicht zusammenhielten. Ich kam auf die Idee, ein 3D-Gehäuse zu entwickeln und es mit einem 3D-Drucker auszudrucken. Dieses Gehäuse habe ich mit der OpenSource Software Blender modelliert [**?**]. Meine Anforderungen an das Gehäuse waren, dass es passgenau ist, die elektronischen Bauteile schützt, die Messungen der Sensoren nicht beeinträchtigt und auch optisch ansprechend ist.

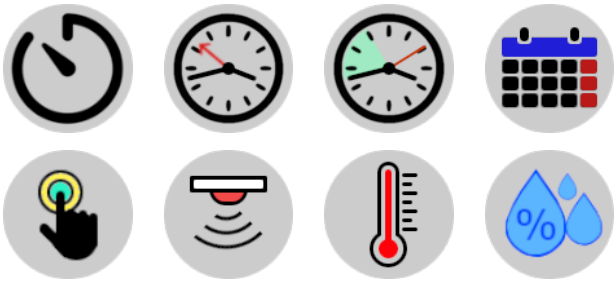


Abbildung 4.2: *Übersicht aller Symbole der Schaltereignisse (von oben links nach unten rechts): Timer, Zeitpunkt, Zeitspanne, Kalender (Wochentage oder Intervall), Knopf (auf dem Display), Bewegung, Temperatur, Luftfeuchtigkeit*

4.1 Die Bedienungsoberfläche

Der Aufbau der Bedienungsoberfläche wird in diesem Abschnitt genauer erklärt.

Eingabe der gewünschten Daten: Um den tPlug einzustellen, stehen verschiedene Sensor- und Zeitereignisse zur Verfügung. Diese Schaltereignisse werden durch kleine Symbole dargestellt, z.B. ein Thermometersymbol für ein Temperaturereignis. Alle Symbole habe ich selbst mit der OpenSource Software Gimp [?] entworfen und sind in Abbildung **??** gezeigt. Durch Klicken kann man dann das richtige Symbol und damit das gewünschte Schaltereignis hinzufügen. Wenn man das richtige Schaltereignis ausgewählt hat, muss es auf den eigenen Bedarf angepasst werden. Zum Beispiel möchte man ein ausgewähltes Temperaturereignis so einstellen, dass es bei einer Temperatur unter 20°C schaltet. Das geht, indem man lange auf das Symbol des Ereignisses drückt. Dadurch öffnet sich ein Popup-Fenster, das jeweils individuell an das ausgewählte Schaltereignis angepasst ist. Dort kann man dann die Werte ändern und durch den Button OK die Änderungen bestätigen.

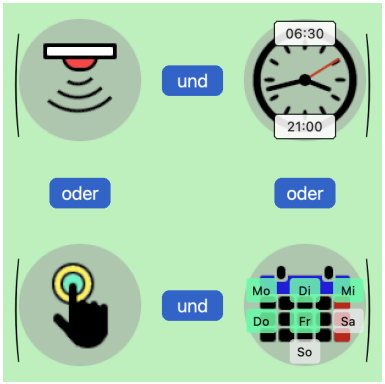


Abbildung 4.3: *Programmierung des tPlugs auf einem 2x2 Raster mit verschieden verknüpften Schaltereignissen*

Da man oft mehrere Ereignisse verknüpfen möchte, wählt man die Symbole in einem 2x2 Raster aus und kann anschließend die ausgewählten Sensoren mit logischen

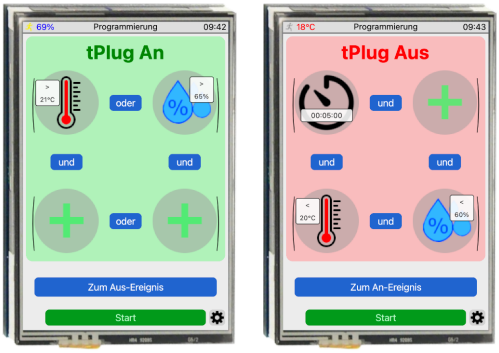


Abbildung 4.4: *Beispielprogrammierung für einen Lüfter:*
Der tPlug geht bei einer Temperatur über 21°C oder einer Luftfeuchtigkeit über 65% an. Er geht wieder aus wenn 5 Minuten vergangen sind und die Temperatur kleiner als 20°C ist, sowie die Luftfeuchtigkeit unter 60% gesunken ist.

und und *oder* Operatoren verknüpfen. Die horizontalen Verknüpfungen werden immer zuerst beachtet, sie sind deshalb optisch eingeklammert auf dem Display dargestellt (siehe Abbildung **??**). Selbstverständlich müssen nicht alle vier Felder belegt werden. Es gibt zwei verschiedene Eingabeoberflächen, eine für *tPlug An* und eine für *tPlug Aus*. Auf beiden ist das oben beschriebene 2x2 Raster zu sehen. Bei *tPlug An* stellt man ein, wann der tPlug angehen soll. Bei *tPlug Aus* stellt man ein, wann der tPlug ausgehen soll.
In Abbildung **??** ist eine mögliche Programmierung des tPlugs für eine Lüftung im Haus gezeigt.
Die zuletzt vorgenommene Einstellung des tPlugs wird automatisch gespeichert, sodass auch nach Trennung von der Netzspannung die vorgenommenen Einstellungen erhalten bleiben. Im Menüpunkt *Einstellungen* können alle eingestellten Schaltereignisse auf einmal zurückgesetzt werden.
Mehr Details sind in der von mir geschriebenen Bedienungsanleitung zu finden (siehe Anhang **??**).

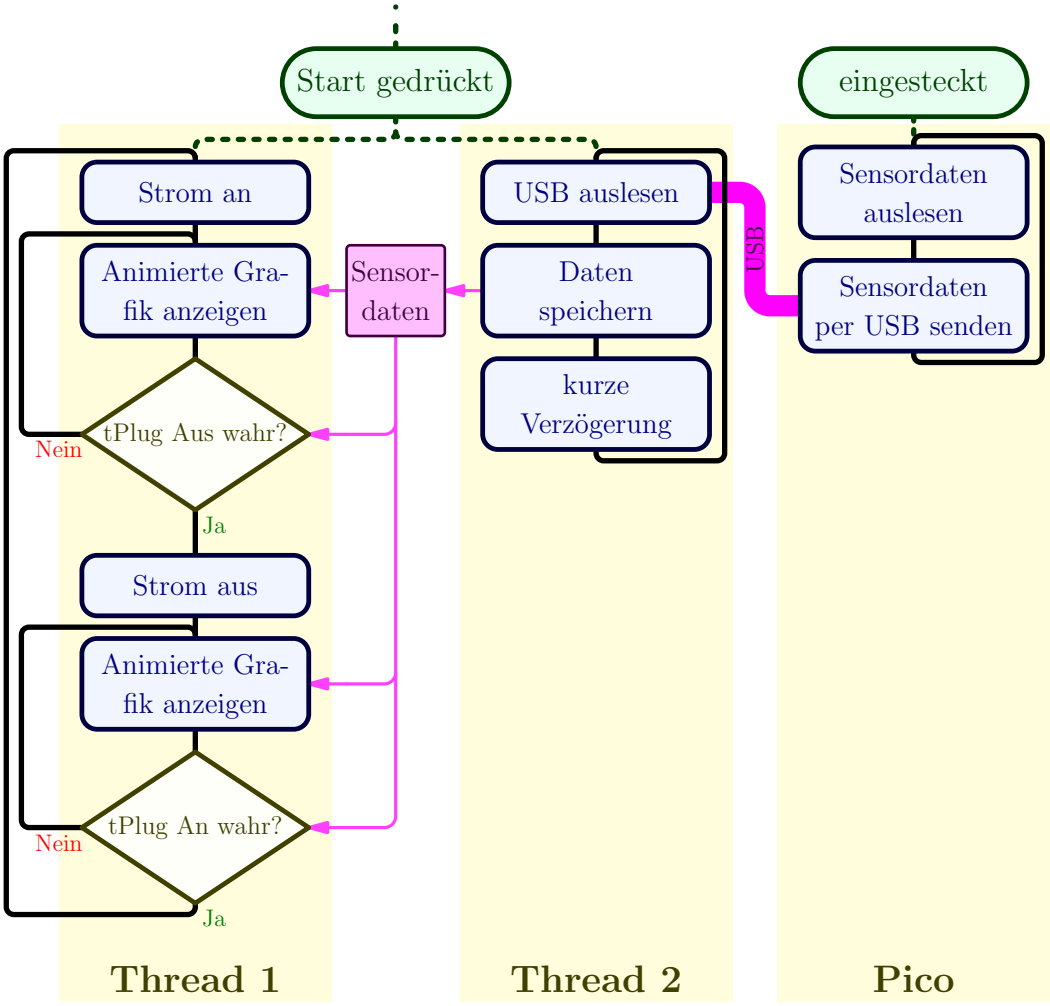


Abbildung 4.5: *Die Abbildung zeigt den Programmablauf, nachdem auf dem TouchDisplay Start gedrückt wurde und wie dieser mit dem Raspberry Pi Pico kommuniziert. Der Raspberry Pi Pico schickt periodisch alle Messwerte der Sensoren per USB an den Raspberry Pi Zero. Der Raspberry Pi Zero hat zwei Threads, das heißt zwei Programmteile die parallel ausgeführt werden. Der eine empfängt die Daten, die per USB geschickt werden und speichert sie in Variablen. Der andere verknüpft die Messwerte mit den eingestellten Werten, schaltet den Strom an bzw. aus und kümmert sich um die Grafik.*

Datenverarbeitung nach Aktivierung: Nachdem der tPlug eingestellt wurde, drückt man auf *Start*. Zu diesem Zeitpunkt besteht das Programm aus zwei parallel ablaufenden Teilen (Threads), das schematisch in Abbildung **??** dargestellt ist. Die Software generiert dann automatisch sowohl für *tPlug An*, als auch für *tPlug Aus* eine Grafik. Auf beiden Grafiken ist ein Wasserkreislauf zu sehen. Auf dem Display ist unten ein Wasserrad, das den Status des tPlugs anzeigt: Wenn es sich dreht ist der tPlug an, wenn nicht dann ist er aus. Zusätzlich wird der Status durch eine grüne LED neben dem Display angezeigt. Im Wasserkreislauf sind die ausgewählten Sensoren mit ihren Symbolen und einem Schieber dargestellt. Der Schieber kann das jeweilige Rohr des Wasserkreislaufes schließen bzw. öffnen. Durch geschickte Abzweigungen und Verkettungen der Schieber, wird die eingestellte Programmierung übersichtlich gezeigt. Zusätzlich ist unter jedem Symbol die jeweils passende Information dargestellt. So sieht man zum Beispiel, wann der Timer abläuft oder wie hoch die aktuelle Luftfeuchtigkeit ist.

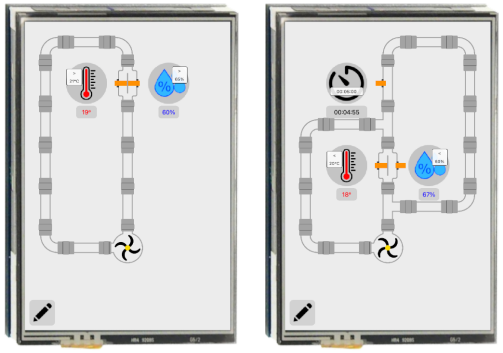


Abbildung 4.6: *Beispiel Wasserkreislauf für einen Lüfter*

Die Steckdose ändert immer ihren Status, wenn *tPlug An* bzw. *tPlug Aus* wahr wird: Bei *tPlug An* geht der Strom an, bei *tPlug Aus* geht der Strom aus. Die beiden Grafiken für die Programmierung aus Abbildung **??** sind in Abbildung **??** dargestellt.
Findet über einen längeren Zeitraum keine Berührung des TouchDisplays statt, verdunkelt sich das Display zum Energiesparen und wird erst nach Berührung wieder aktiviert.

4.2 Elektronik des tPlugs

Die Elektronik des tPlugs besteht aus zwei Mikrocontrollern. Der eine Mikrocontroller (Raspberry Pi Pico) steuert die Sensoren und schickt deren Messwerte über ein USB

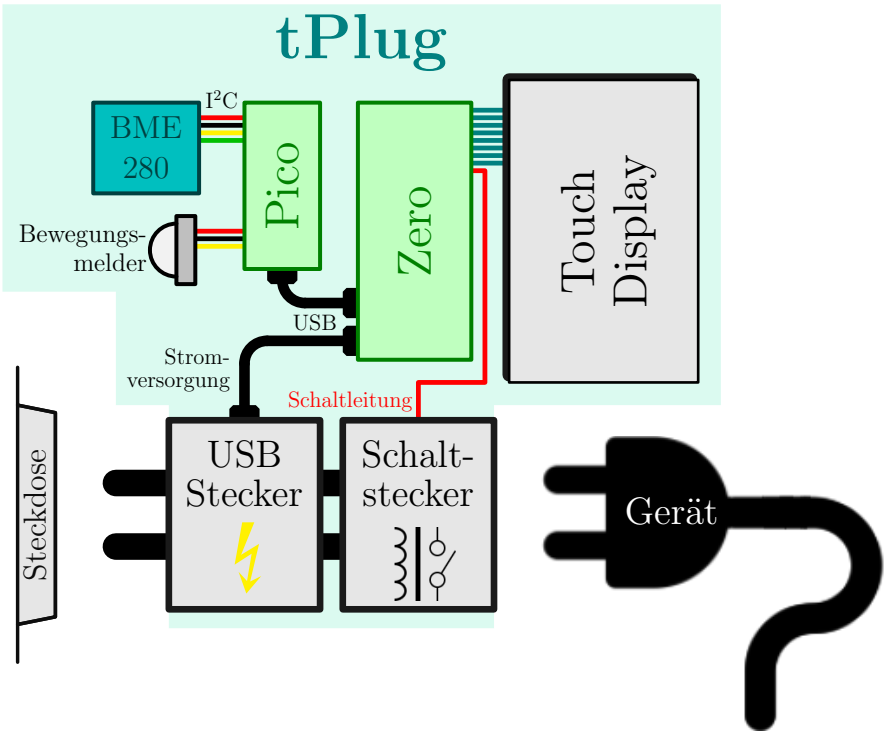
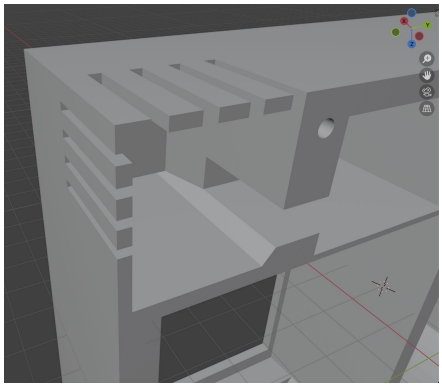


Abbildung 4.7: *Die Abbildung zeigt schematisch den Aufbau der Hardware.*

Kabel zum zweiten Mikrocontroller, dem Raspberry Pi Zero. Dieser steuert das Touch-Display an und schaltet den Strom an und aus. Der Aufbau der gesamten Hardware ist schematisch in Abbildung **??** dargestellt.

Die Sensoren und deren Mikrocontroller: Im tPlug verwende ich verschiedene Sensoren, damit der Schalter flexibel (auch abhängig von der Umgebung) eingesetzt werden kann. Ich habe einen Bewegungsmelder und einen Umweltsensor (BME 280 [**?**]) eingebaut. Der Umweltsensor misst Temperatur, Luftfeuchtigkeit und Luftdruck. Für den tPlug sind jedoch nur Temperatur und Luftfeuchtigkeit relevant, da der Luftdruck im Alltag eigentlich keine Rolle spielt. Alle Sensoren werden von einem Mikrocontroller, dem Raspberry Pi Pico, ausgelesen. Der Umweltsensor wird über I^2C angesteuert. Der Bewegungsmelder wird über die GPIOs des Picos ausgelesen. Die Daten von beiden Sensoren werden über USB durch ein von mir selbst entwickeltes USB-Protokoll in Echtzeit weitergeleitet.

Das TouchDisplay und der zentrale Mikrocontroller: Das TouchDisplay ist ein wichtiger Teil des tPlugs. Über das TouchDisplay werden Einstellungen vorge-



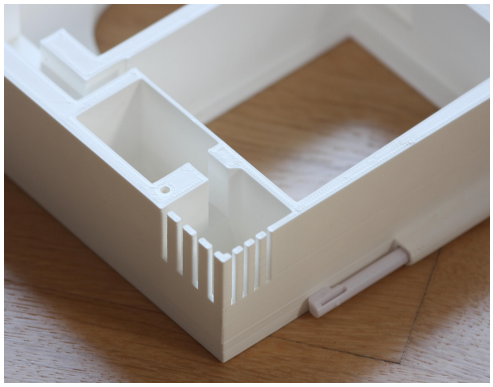


Abbildung 4.8: *Die Abbildung links zeigt die mit Blender modellierte 3D-Form des Deckgehäuses. Gezeigt ist der Ausschnitt der Kammer für den Umweltsensor, der durch die Wände thermisch von den Mikrcontrollern abgeschirmt ist und durch die Schlitze die Umgebungsparameter zuverlässig messen kann. Im Bild rechts ist dieser Ausschnitt auf der neuesten Version des 3D-gedruckten Deckgehäuses des Prototyps zu sehen.*

nommen und die aktuellen Messwerte dargestellt. Das TouchDisplay wird auch über I^2C [**?**] und GPIOs [**?**] von einem Raspberry Pi Zero angesteuert. Der Raspberry Pi Zero wird über einen USB-Anschluss mit Strom versorgt und schaltet das Gerät über die Schaltsteckdose per Relais an und aus. Über ein weiteres USB-Kabel empfängt der zentrale Mikrocontroller die Sensordaten vom Raspberry Pi Pico.

Der Vorteil von zwei unabhängigen Mikrocontrollern: Es gibt mehrere Gründe, wieso ich zwei Mikrocontroller benutze. Zum einen lässt sich das TouchDisplay direkt auf die Pins des Raspberry Pi Zeros stecken. Für die Ansteuerung des Umweltsensors hätte ich aber Pins gebraucht, die auch das TouchDisplay nutzt. Ich hätte die Pins modifizieren müssen, um beide Komponenten anschließen zu können. Dieses Problem kann ich durch die Nutzung von zwei Mikrocontrollern umgehen. Ein zweiter Grund ist, dass ich durch die zwei Mikrocontroller die Sensor- und Displayansteuerung unabhängig programmieren und testen konnte. Die zur Testung des Programms benötigten Messwerte habe ich am Computer durch Zufallsgeneratoren simuliert.

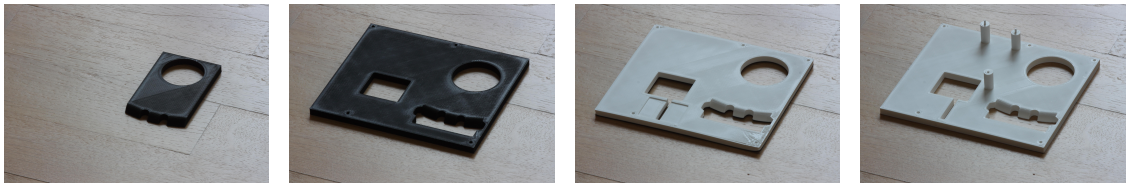


Abbildung 4.9: *In dieser Abbildung sind die Entwicklungsschritte der Mittelplatte dargestellt. Ganz links modellierte ich zunächst die Form des Relais-Steckers. Ausgehend von diesem Teilstück, passte ich schrittweise die Mittelplatte bis zur passgenauen Form ganz rechts an, in der nun Aussparungen für Kabel und Halterstifte für das Touch-Display eingearbeitet sind.*

4.3 Das Gehäuse des tPlugs

Damit der tPlug wirklich als ein funktionales Haushaltsgerät verwendet werden kann, entwickelte ich eine robuste Hülle, die alle Teile zusammenhält und die Sensoren nicht beeinflusst. Die entworfene Hülle besteht aus drei Teilen (Deckgehäuse, Mittelplatte und Rückenteil), die nach Einsetzen der Bauteile miteinander verschraubt werden. Damit der Temperatur- und Feuchtigkeitssensor zuverlässig die Umgebungstemperatur und -feuchtigkeit messen kann, habe ich den Umweltsensor hinter eine Wand mit Schlitzen eingebaut. Um zu vermeiden, dass die Abwärme der Mikrocontroller das Messergebnis des Umweltsensors beeinflusst, musste ich den Sensor zusätzlich thermisch abschirmen. Daher habe ich ihn in einer separaten Kammer in der Hülle verbaut (siehe Abbildung **??**).

Herausfordernd war außerdem das TouchDisplay so im Gehäuse zu befestigen, dass es während der Bedienung durch den Druck nicht nach innen rutscht. Um dies zu vermeiden, konstruierte ich an der Mittelplatte Stifte, die das TouchDisplay fixieren. Die gesamte Hülle habe ich in Blender modelliert und anschließend mit dem 3D-Drucker ausgedruckt. Die Entwicklung war sehr aufwändig und ich habe einige Testdrucke benötigt, vor allem weil ich die 3D-Hülle passgenau auf die teils unförmigen Bauteile (vorgegebene Rundungen am Stecker) anfertigen musste. Zuletzt ergänzte ich noch einen Stifthalter an der Hülle. Einige Entwicklungsschritte der von mir entwickelten Mittelplatte sind in der Fotoreihe **??** zu sehen.

5 Ergebnisse

Nach stetiger Verbesserung der Bedingungsoberfläche und des Gehäuses bin ich mit dem Prototyp des von mir konstruierten Schaltsteckers tPlug sehr zufrieden. Ich probierte die verschiedenen Funktionen des tPlug in unterschiedlichen Alltagsanwendungen erfolgreich aus.
Damit der tPlug auch von anderen Personen verwendet werden kann, erstellte ich eine Bedingungsanleitung, die im Anhang **??** zu finden ist.
Um über die Alltagstauglichkeit und die Akzeptanz der Funktionalität und Bedienungsoberfläche von anderen Personen Rückmeldung zu bekommen, habe ich den Prototyp von Anwendern in unterschiedlichen Altersklassen ausprobieren lassen. Zunächst stellte ich als vorgegebene Aufgabe, den tPlug mit Hilfe der Bedienungsanleitung so einzustellen, dass der Bewegungsmelder ein Licht für eine Zeit von einer Minute einschalten sollte. Mit dieser Aufgabenstellung haben den tPlug drei Personen im Alter von über 65 Jahren, drei Personen im Alter zwischen 25 und 64, und eine Personen unter 25 Jahren getestet. Die Versuchspersonen unter 65 Jahren kamen problemlos mit dem Schaltstecker zusammen mit der Anleitung zurecht. Die Versuchspersonen über 65 Jahren konnten mit der Bedienungsoberfläche des Schaltstecker nach kurzer Demonstration gut umgehen und dann auch die vorgegebene Aufgabe lösen. Anschließend sollten die Versuchspersonen noch einige Anwendungsideen am tPlug ausprobieren und einstellen. So sind auch noch weitere Anwendungsideen, wie beispielsweise temperaturabhängige Bewässerungsanwendungen entstanden.
Außerdem konnte ich von den Testpersonen noch hilfreiche Hinweise und Tipps zur Verbesserung der Software bekommen, die ich dann direkt umgesetzt habe. Beispielsweise haben viele Tester den Knopf *Zum An-Ereignis* bzw. *Zum Aus-Ereignis* nicht auf Anhieb gefunden. Deswegen habe ich inzwischen den Knopf optisch deutlich sichtbarer in blau dargestellt. Außerdem habe ich noch ein paar weitere Verbesserungsvorschläge zur Visualisierung der Statusanzeige während des Schaltvorgangs des tPlugs bekommen, die zu einer grafischen Verbesserung des dargestellten Rohrsystem geführt haben.

6 Umgesetzte Erweiterungen

Um die Einsatzmöglichkeiten des tPlugs weiter zu erweitern, kann man den Schaltstecker nun auch über Wi-Fi mit dem Heimnetzwerk verbinden. Dazu programmierte ich ein Auswahlmenü, in dem die verfügbaren Wi-Fi-Netzwerke gesucht, angezeigt und ausgewählt werden können. Zur Verbindungsherstellung mit dem Netzwerk musste ich ein On-Board-Keyboard programmieren, damit auch das Wi-Fi-Passwort eingegeben werden kann. Diese neue Wi-Fi-Funktionalität kann in den Einstellungen aktiviert und deaktiviert werden. Außerdem entwickelte ich ein Web-Interface, mit dem der Status des tPlugs über einen Webbrowser in Echtzeit angezeigt wird. Die Daten übertrage ich mit einem in C++ programmierten WebSocket. Auf dem tPlug richtete ich einen Webserver ein, auf den die Benutzer mittels Browser zugreifen können. Die Webseite realisierte ich mit JavaScript, HTML und CSS. (Stand 1. März 2026)

7 Fazit und Ausblick

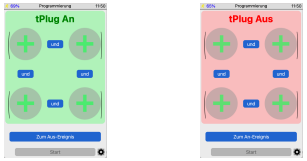
Durch den Testbetrieb im Alltag und die Testpersonen haben sich noch andere Erweiterungsmöglichkeiten sowohl in der Software als auch in der Hardware des tPlugs ergeben. Zunächst einmal wäre es nützlich, verschiedene Einstellungen des tPlugs zu speichern, so dass zwischen verschiedenen Anwendungsmöglichkeiten eines Schaltsteckers schnell gewechselt werden kann. Derzeit kann über Wi-Fi lediglich der Status des tPlugs angezeigt werden. Um eine Ansteuerung aus der Ferne zu ermöglichen, plane ich das ganze Gerät über das Web-Interface steuern zu können. Dafür müsste ich aber noch stärker Sicherheitsaspekte berücksichtigen, sodass das Gerät nicht von Fremden fehlgesteuert werden kann.
Die Hardware des tPlugs möchte ich durch weitere Sensoren erweitern. Beispielsweise würde ich gerne über einen USB-Anschluss bzw. über Bluetooth weitere externe Messgeräte mit dem tPlug verbinden. Zum Beispiel könnte ein über Bluetooth verbundener Bodenfeuchtigkeitssensor für die Bewässerung des Gartens oder von Zimmerpflanzen neue interessante Einsatzmöglichkeiten eröffnen.

Anleitung tPlug

EINSCHALTEN
Der tPlug startet automatisch nach Einstecken in die Steckdose.

WAHL DER SCHALTEREIGNISSE
Zur Programmierung gibt es zwei Bedienungsoberflächen:

tPlug An | tPlug Aus



Wechseln zwischen den Bedienungsoberflächen durch Klicken des blauen Knopfs *Zum An-Ereignis* bzw. *Zum Aus-Ereignis*.
Auf den Oberflächen sind auf einem 2x2 Raster vier Platzhalter (Plus-Symbol) zu sehen. Diese können durch wiederholtes Klicken mit Schaltereignissen gefüllt werden.
Folgende Schaltereignisse stehen zur Verfügung:



HINWEIS:
Es müssen nicht alle Platzhalter mit Schaltereignissen besetzt werden. Plus-Symbole werden beim Schaltprozess nicht beachtet.

VERKNÜPFUNG DER SCHALTEREIGNISSE
Die Schaltereignisse in den beiden Oberflächen *tPlug An* und *tPlug Aus* werden durch ein *und* oder *oder* verknüpft. Durch Klicken auf die blauen und/oder Knöpfe, kann man zwischen *und* und *oder* wechseln (siehe oben). Dabei werden die zwei Schaltereignisse in den beiden Reihen jeweils zuerst verknüpft und anschließend die beiden Reihen selbst.

EINSTELLUNG DER SCHALTEREIGNISSE
Wenn man ein gewünschtes Schaltereignis durch Klicken ausgewählt hat, kann man durch langes Drücken auf das Symbol das Schaltereignis einstellen. Es öffnet sich ein spezifisches Einstellungsfenster. Dieses sieht beispielsweise für die Schaltereignisse *Zeitspanne* und *Kalender* wie folgt aus:

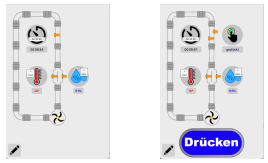
Zeitspanne | Kalender





Nach erfolgter Einstellung drückt man *OK*, andernfalls *Abbrechen*.

START
Nach erfolgter Programmierung wird der Schaltprozess durch Drücken des grünen Start-Knopfs am unteren Rand des Fensters gestartet.
Der Status des tPlugs wird durch ein „Rohrsystem“ angezeigt, welches beispielsweise wie folgt aussieht:



Die orangenen Schieber zeigen den Status der einzelnen Schaltereignisse zusätzlich mit der passenden Information an.
Wurde das Schaltereignis *Knopf* ausgewählt, erscheint unten auf dem Display ein blauer Knopf zur Schaltung dieses Schaltereignisses.
Dreht sich das Wasserrad, so ist der Strom gerade angeschaltet. Sobald der Wasserkreislauf durch Schieber unterbrochen wurde, geht der Strom aus. Dann ändert sich das Rohrsystem und der Wasserkreislauf muss wieder geöffnet werden.
Der Status des Steckers (An/Aus) wird zusätzlich durch die Grüne LED neben dem Display angezeigt.
Durch das Stiftsymbol unten links gelangt man wieder in die Einstellungsebene.

HINWEIS:
Der Startknopf kann nur gedrückt werden, wenn sowohl bei *tPlug An* als auch bei *tPlug Aus* mindestens ein Schaltereignis ausgewählt ist.

EINSTELLUNGEN
Zur Oberfläche *Einstellungen* gelangt man durch Klicken auf das Einstellungssymbol im Programmiermodus unten rechts.

Literatur

[1] *GESAS GmbH*, Resisitve und Kapazitive TouchDisplays

[2] *UM10204*, I^{2}C-bus specification and user manual,
https://www.nxp.com/docs/en/user-guide/UM10204.pdf

[3] *Christian Immler*, Erste Schritte mit dem Raspberry Pi, 2019 Franzis Verlag.

[4] *User Experience, Usability & Interface Design Blog*,
10 Prinzipien für Usability und UI-Design

[5] *Gundolf Haase*, Einführung in die Programmierung mit C++,
Skript Sommersemester 2025, 25. November 2025 – Universität Graz

[6] *Wikipedia*, Objektorientierte Programmierung,
https://de.wikipedia.org/wiki/Objektorientierte_Programmierung

[7] *Qt for Open Source Development*,
https://www.qt.io/development/download-open-source

[8] *Passiver Infrarot-Bewegungssensor SEN-HC-SR501*, Datenblatt SEN-HC-SR501

[9] *Wikipedia*, Thermistor,
https://de.wikipedia.org/wiki/Thermistor

[10] *Wikipedia*, Hygrometer,
https://de.wikipedia.org/wiki/Hygrometer

[11] *Umweltsensor BME 280*, Datenblatt BME 280

[12] *OpenSource Software Blender*, https://www.blender.org

[13] *OpenSource Software Gimp*, https://www.gimp.org